MÉTRICAS E ESTIMATIVA DE SOFTWARE

AULA 3

Profª Rosemari Pavan Rattmann

CONVERSA INICIAL

Nesta etapa, exploraremos os fundamentos dos métodos de estimativa de *software*, avaliando diversas abordagens essenciais para a prática eficaz de estimativa em projetos de desenvolvimento de *software*.

Focaremos inicialmente no processo amplo de estimativa de *software*, destacando a adaptação dos métodos tradicionais às metodologias ágeis. Em particular, examinaremos o *Planning Poker*, uma técnica colaborativa que visa envolver toda a equipe no processo de estimativa, promovendo o consenso e a precisão das estimativas em um ambiente ágil.

Trataremos então dos modelos baseados em pontos de função, uma abordagem amplamente utilizada para medir o tamanho funcional de um sistema de *software*, independentemente da tecnologia utilizada para sua implementação. Analisaremos como os pontos de função podem ser empregados na estimativa de esforço e custo, fornecendo uma métrica valiosa para a gestão de projetos.

Abordaremos também o *Constructive Cost Model* (COCOMO), um modelo clássico de estimativa de esforço amplamente adotado na indústria de *software*. Investigaremos suas diferentes variantes e como elas podem ser aplicadas para estimar o esforço e os recursos necessários em projetos de desenvolvimento de *software*.

Aprofundaremos nosso estudo ao analisar os modelos de estimativa baseados na NESMA (Netherlands Software Metrics Association), uma organização que promove a aplicação de métricas de *software* para melhorar a gestão e qualidade de projetos de *software*. Veremos como esses modelos podem complementar e enriquecer as abordagens tradicionais de estimativa.

Por fim, dedicaremos um tempo para comparar e contrastar os diferentes modelos de estimativa de *software* abordados nesta etapa. Examinaremos suas vantagens, limitações e casos de uso ideais, permitindo a você uma compreensão abrangente das opções disponíveis e como selecionar o modelo mais adequado para cada contexto de projeto.

Ao explorar tais temas, estaremos preparados para enfrentar os desafios complexos associados à estimativa de *software* e tomar decisões fundamentadas em suas práticas de desenvolvimento. Vamos lá?

TEMA 1 – PROCESSO DE ESTIMATIVA DE *SOFTWARE* MAIS ÁGEIS

Estudaremos os princípios fundamentais dos métodos de estimativa de *software*, com foco na adaptação desses métodos às metodologias ágeis. Exploraremos o *Planning Poker*, uma técnica colaborativa que busca envolver toda a equipe no processo de estimativa, promovendo consenso e precisão.

A medição de *software* tem como propósito quantificar atributos essenciais de um produto ou processo de desenvolvimento de *software* (Sommerville, 2021). Desde os anos 1950, a estimativa de esforço no desenvolvimento de *software* tem sido uma prática consolidada, tornando-se uma necessidade fundamental para empresas do ramo, uma vez que a estimativa de esforço serve como um meio para avaliar o custo total envolvido na execução de um projeto ou produto, além de sua qualidade.

Essas estimativas de esforço e custo geralmente se baseiam na previsão do tamanho do sistema a ser desenvolvido. Contudo, em projetos de melhoria (evolução ou manutenção), o tamanho do sistema pode servir como uma referência para calcular o esforço necessário da equipe. Por meio dessas estimativas, é possível ter uma noção do tamanho e do tempo necessários para o desenvolvimento ou evolução de um sistema, o que, por sua vez, define o escopo do projeto (Wanderley, 2015). Uma estimativa de custo e esforço elevada pode indicar o grande tamanho de um *software*, enquanto uma estimativa baixa pode sugerir que o projeto é de menor escala.

Em projetos caracterizados por mudanças frequentes, a necessidade de refatoração do código é inevitável. Esse cenário é ainda mais desafiador em equipes reduzidas e com prazos de entrega apertados, tornando a elaboração de um planejamento eficaz uma tarefa árdua.

A abordagem ágil surge como uma resposta a esses desafios, priorizando o desenvolvimento iterativo, incremental e a colaboração constante entre os membros da equipe. Em um ambiente onde as mudanças são a norma, a metodologia ágil se adapta facilmente, buscando ajustar-se continuamente às novas exigências.

Para aumentar as chances de sucesso de um projeto (Pressman, 2016), é essencial empregar métricas que permitam monitorar o desenvolvimento, fornecendo estatísticas e comparações que possibilitem uma avaliação precisa

do progresso. A medição, nesse sentido, envolve a coleta de dados para o cálculo das métricas e a análise dos resultados obtidos.

A utilização de estimativas é fundamental nesse processo, possibilitando a obtenção de métricas que permitem prever a quantidade de recursos humanos necessários, o tempo requerido e os custos envolvidos no desenvolvimento do projeto. Essas estimativas oferecem uma visão antecipada das demandas deste, facilitando o planejamento e a tomada de decisões ao longo do processo de desenvolvimento.

A seguir, vamos analisar como estimar métricas para o desenvolvimento de projetos que utilizam metodologias ágeis. É um assunto relativamente novo, dado que a grande maioria das empresas utiliza estimativas tradicionais para mensuração de custos e tempos. Nesse sentido, examinaremos três técnicas de estimativa: *Ideal Day*, *Planning Poker* e Ponto de Função.

1.1 Técnica *Ideal Day*

A técnica de estimativa conhecida como *Ideal Day* é uma ferramenta valiosa em metodologias ágeis, como o *Scrum*, para determinar o tempo necessário para a realização de tarefas durante o planejamento de um *sprint*. Essa abordagem parte do princípio de que um "dia ideal" representa um período de trabalho sem interrupções ou distrações, permitindo que os desenvolvedores se concentrem completamente na atividade em questão.

No processo de aplicação do *Ideal Day*, a equipe de desenvolvimento precisa compreender claramente o escopo de cada tarefa a ser estimada. Durante o planejamento do *sprint*, cada membro da equipe é convidado a fornecer uma estimativa do tempo que levaria para completar sua parte da tarefa, expressando-a em dias ideais de trabalho. Isso implica em imaginar quanto tempo seria necessário para concluir a tarefa em condições ideais, sem considerar imprevistos ou interrupções.

É essencial esclarecer que o conceito de *Ideal Day* não se refere a um dia de trabalho prolongado, mas a um período realista e sustentável para o desenvolvedor. Portanto, a equipe precisa estabelecer uma definição clara do que constitui um "dia ideal" em seu contexto, levando em conta fatores como jornada de trabalho, pausas e outras atividades não relacionadas à tarefa em si.

Após todas as estimativas individuais serem fornecidas, elas são somadas para determinar a estimativa total de tempo necessário para concluir

todas as tarefas do *sprint*. Essa estimativa agregada é então usada para planejar a capacidade do *sprint*, permitindo que a equipe determine quantas tarefas podem ser incluídas com base na capacidade de trabalho disponível.

Embora a técnica do *Ideal Day* seja uma ferramenta simples e compreensível, é importante lembrar que as estimativas baseadas nessa abordagem são apenas previsões e podem variar na prática. Portanto, a equipe deve revisar continuamente suas estimativas com base no progresso real do trabalho durante o *sprint*, adaptando-as conforme necessário para garantir maior precisão ao longo do tempo. Essa prática de revisão e ajuste constantes das estimativas é uma característica central das metodologias ágeis, permitindo uma adaptação contínua às mudanças e uma melhoria na precisão das estimativas ao longo do tempo.

1.2 Técnica *Planning Poker*

O *Planning Poker* é das técnicas mais utilizadas em metodologias ágeis para estimar o esforço necessário para completar as tarefas durante o planejamento do *sprint*. Essa abordagem colaborativa visa envolver toda a equipe na estimativa, promovendo o consenso e aumentando a precisão das estimativas.

O processo do *Planning Poker* geralmente começa com uma reunião da equipe de desenvolvimento, na qual as histórias de usuário ou tarefas a serem estimadas são apresentadas. Cada membro da equipe recebe um conjunto de cartas numeradas, geralmente representando valores como 0, 1, 2, 3, 5, 8, 13, 20, 40 e 100, que correspondem a unidades de esforço, como pontos de história, ou horas ideais de trabalho.

À medida que cada história é apresentada, os membros da equipe selecionam a carta que melhor representa o esforço que eles acreditam ser necessário para completar a tarefa. No entanto, é importante que essas seleções sejam feitas individualmente e sem influência dos outros membros da equipe.

Uma vez que todos tenham escolhido suas cartas, elas são reveladas simultaneamente. Se houver uma grande discrepância entre as estimativas, os membros da equipe têm a oportunidade de discutir os diferentes pontos de vista e os fatores que influenciaram suas escolhas. Essa discussão ajuda a equipe a entender melhor a complexidade da tarefa e a alinhar suas expectativas.

Após a discussão, a equipe repete o processo, selecionando novamente as cartas e revelando-as simultaneamente. Esse ciclo de estimativa e discussão continua até que a equipe chegue a um consenso sobre a estimativa de esforço para a tarefa em questão.

O *Planning Poker* é uma técnica eficaz porque incentiva a participação de toda a equipe, aproveitando a experiência e o conhecimento de cada membro. Além disso, promove a transparência e a comunicação aberta, permitindo que a equipe identifique e resolva rapidamente quaisquer divergências nas estimativas. Essa abordagem colaborativa resulta em estimativas mais precisas e em um planejamento mais eficaz do *sprint*.

TEMA 2 – MODELOS BASEADOS EM PONTOS DE FUNÇÃO

Vamos explorar agora os modelos baseados em pontos de função, uma abordagem amplamente adotada para avaliar o tamanho funcional de um sistema de *software*, sem depender da tecnologia usada em sua implementação. Vamos examinar como os pontos de função podem ser utilizados para estimar o esforço e os custos, fornecendo uma métrica valiosa para o gerenciamento de projetos.

2.1 Técnica de Pontos de Função

Análise de Pontos de Função (APF, ou, em inglês, FPA), é uma técnica de medição de *software* utilizada para quantificar o tamanho funcional de um sistema de *software* com base nas funcionalidades fornecidas ao usuário final. Pontos de Função (PF) medem o tamanho funcional do *software*, cujo tamanho funcional pode ser definido como uma medida de tamanho de *software*; essa medida é realizada a partir da funcionalidade implementada em um sistema sob o ponto de vista do usuário.

Esta técnica, regulamentada pela *International Function Point Users Group* – IFPUG ("Grupo Internacional de Usuários de Pontos de Função"), estabelece padrões e diretrizes para uma medição precisa do tamanho funcional do *software* e é a técnica mais amplamente utilizada comercialmente nessa indústria. A IFPUG é responsável por manter e atualizar a *Counting Practices Manual* (CPM, ou "Manual de Práticas de Contagem"), um documento abrangente que define as regras e metodologias para contar pontos de função

de acordo com os padrões estabelecidos. Esse manual é um guia indispensável para os praticantes da APF em todo o mundo, garantindo uma interpretação uniforme e confiável das diretrizes da técnica.

A padronização proporcionada pela IFPUG e pela CPM garante uma interpretação consistente e uniforme da técnica, promovendo a comparabilidade e a transparência entre projetos e organizações. Isso é especialmente importante em um ambiente comercial no qual a precisão e a confiabilidade das estimativas são fundamentais para o sucesso do projeto e para a tomada de decisões estratégicas.

A popularidade da APF no ambiente comercial pode ser atribuída a várias razões. Em primeiro lugar, a APF oferece uma abordagem objetiva e independente para medir o tamanho funcional do *software*, o que a torna uma métrica valiosa para empresas e clientes. Ao contrário de medidas baseadas em linhas de código, que podem variar dependendo da linguagem de programação ou estilo de codificação, a APF se concentra nas funcionalidades entregues ao usuário final, proporcionando uma visão mais precisa e confiável do tamanho do *software*. Além disso, a APF é altamente adaptável e aplicável a uma ampla gama de domínios e tipos de projetos de *software*. Em um sistema de gestão de banco de dados, em um aplicativo web, ou em um *software* embarcado, a APF pode ser utilizada para medir o tamanho funcional e apoiar o processo de estimativa, planejamento e controle em todos os estágios do ciclo de vida do desenvolvimento de *software*.

Os principais tipos de medidas que podem ser feitas com a APF, são:

- **Medição total**: a medida total é o resultado final da contagem de pontos de função para um sistema de *software* pronto. Envolve identificar e classificar todas as funcionalidades do sistema, atribuir pontos de função a cada funcionalidade com base em sua complexidade e calcular o total de pontos de função para o sistema como um todo.
- **Estimativa**: a APF também pode ser utilizada para realizar estimativas do tamanho funcional de um *software* antes mesmo de ele ser construído. Isso é feito com base em especificações de requisitos ou em análises preliminares do sistema. As estimativas de pontos de função fornecem uma previsão do tamanho do *software*, o que é útil para o planejamento inicial do projeto, incluindo estimativas de esforço, custo e prazo. Pode

ser realizada para estimar o esforço e custo de um projeto de melhoria (manutenção).

- **Medição indicativa**: é uma abordagem mais simplificada que foca em um aspecto específico do *software*, como as funcionalidades de armazenamento de dados. Em vez de medir todas as funcionalidades do sistema, a medida indicativa se concentra apenas em um subconjunto de funcionalidades relevantes para uma análise específica. Isso pode ser útil em situações em que seja necessária uma avaliação de viabilidade de desenvolvimento de um novo *software*, quando não se conhecem todos os requisitos refinados.

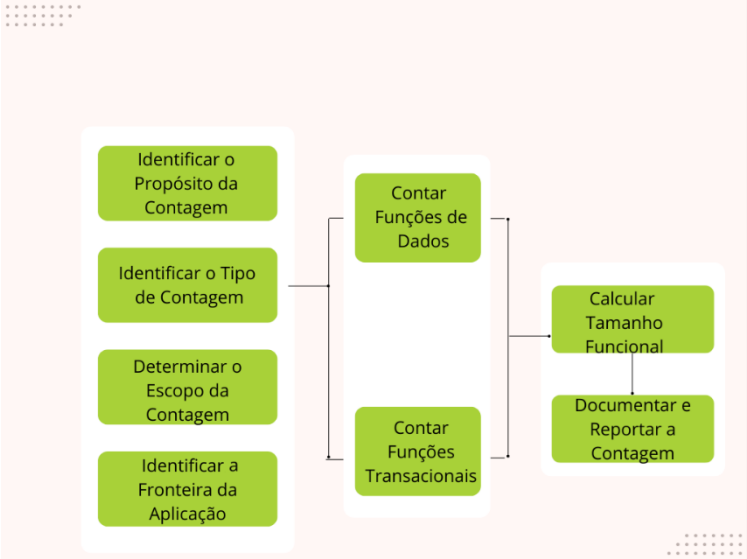
2.3 Processo da análise de pontos de função

A análise de pontos de função é um processo meticuloso e estruturado utilizado para medir o tamanho funcional de um *software*, focando na perspectiva do usuário e nas funcionalidades que ele percebe e utiliza. Esse método oferece uma abordagem objetiva para quantificar as características que um sistema de *software* proporciona aos seus usuários finais, independentemente da tecnologia utilizada (Vazquez, 2003). Estudaremos, a seguir, os conceitos da análise de pontos de função.

- **Os objetivos da análise de pontos de função são:**
  - Medir a funcionalidade implementada no *software*, que o usuário solicita e recebe;
  - Medir a funcionalidade impactada pelo desenvolvimento, melhoria e manutenção de *software*, independentemente da tecnologia utilizada na implementação.

A Figura 1 demonstra o procedimento da contagem por pontos de função.

Figura 1 – Contagem por pontos de função



- **A contagem de pontos de função pode ser feita para:**
  - Um projeto de desenvolvimento (novo): deve medir o tamanho estimado de um *software* que ainda será desenvolvido;
  - Um projeto de melhoria/manutenção: mede as funcionalidades que foram adicionadas ou removidas por meio de um projeto de melhoria;
  - Em uma aplicação (pronta): mede o tamanho de um *software* pronto e instalado.
- **A fronteira da aplicação:** A determinação da fronteira é de suma importância no processo de contagem, pois a determinação errônea desta pode prejudicar o resultado final da medição. De acordo com o CPM, a fronteira é uma interface conceitual entre o *software* sob estudo e seus usuários (Vazquez, 2003). A fronteira da aplicação:
  - Define o que é externo à aplicação;
  - Indica a fronteira entre o *software* que está sendo medido e o usuário;
  - Atua como uma “membrana” através da qual os dados processados pelas transações (EEs, SEs e CEs) passam para dentro e para fora da aplicação;
  - Envolve os dados lógicos mantidos pela aplicação (ALIs);
  - Auxilia na identificação dos dados lógicos referenciados, mas não mantidos pela aplicação (AIEs);
  - Depende da visão externa do negócio do usuário da aplicação,
  - Independente de questões técnicas e/ou implementação.

- **Ponto de vista do usuário:** Um dos princípios da análise de pontos-de-função focaliza-se na perspectiva de como os usuários "enxergam" os resultados que um sistema produz. A visão do usuário representa uma descrição formal das necessidades dos negócios e na linguagem deste, e os desenvolvedores traduzem a informação do usuário para informações em linguagem técnica a fim de prover uma solução. O processo de análise de pontos de função inicia-se com a identificação das funcionalidades do sistema sob o ponto de vista do usuário, incluindo todas as operações que este pode realizar, como inserir dados, visualizar relatórios, buscar informações e realizar outras interações com o *software*. A intenção é entender o que o usuário deseja alcançar e como o sistema o auxilia nesses objetivos.
- **Processo elementar:** Um processo elementar é uma função ou transação elementar identificável dentro de um sistema de *software*. Ele representa uma atividade de negócio ou uma ação direta do usuário que resulta em uma entrada ou saída de dados, ou uma consulta a dados internos. Em essência, um processo elementar é uma unidade indivisível de funcionalidade que pode ser contada e avaliada separadamente para determinar sua complexidade e contribuição para o tamanho funcional total do sistema.
- **Funções de dados:** As funções de dados representam a funcionalidade oferecida ao usuário para satisfazer requisitos de armazenamento de dados internos e externos. Uma função de dados pode ser um arquivo lógico interno ou um arquivo de interface externo. O termo arquivo aqui não se refere a arquivo físico ou tabela, nem à implementação física destes grupos de dados, mas a um grupo de dados logicamente relacionados (Vazquez, 2003).
- **Arquivo Lógico Interno:** Grupo de dados ou informações de controle logicamente relacionados, identificável pelo usuário, mantido dentro da fronteira da aplicação. A intenção primária de um ALI é armazenar dados mantidos por meio de um ou mais processos elementares da aplicação sendo contada.
- **Arquivo de interface externa:** Grupo de dados logicamente relacionados ou informação de controle, reconhecido pelo usuário, referenciado pela aplicação sendo medida, mas que é mantido dentro da fronteira de outra

aplicação. A intenção primária de um AIE é armazenar dados referenciados por um ou mais processos elementares dentro da fronteira da aplicação medida. Isto significa que um AIE contado por uma aplicação deve ser um ALI em outra aplicação.

- **Registro Lógico Referenciado (RLR ou TR):** Entidade de dados referenciada ou usada por um processo elementar dentro do sistema de *software*. Esse registro contém informações necessárias para a execução de uma função específica dentro do contexto do sistema.
- **Funções transacionais:** Uma função de transação é um processo elementar que oferece funcionalidade ao usuário para processar dados. Uma função de transação é uma entrada externa, saída externa, ou consulta externa.
- **Entrada externa (EE):** Processo elementar que processa dados (ou informações de controle) vindos de fora da fronteira da aplicação. A principal intenção de uma EE é manter um ou mais ALI e/ou alterar o comportamento do sistema. A Figura 2 mostra um exemplo de EE.

Figura 2 – Exemplo de entrada externa

Fonte: elaborada por Rattmann, 2024, com base em Costa, S.d.

- **Consulta externa (CE):** Processo elementar que envia dados ou informações de controle para fora da fronteira da aplicação. A principal intenção de uma CE é apresentar informação ao usuário por meio de uma simples recuperação de dados ou informações de controle de um ALI ou AIE. A Figura 3 mostra um exemplo de CE.

Figura 3 – Exemplo de consulta externa

Fonte: elaborada por Rattmann, 2024, com base em Costa, S.d.

- **Saída externa (SE):** Processo elementar que gera dados ou informações de controle que saem pela fronteira da aplicação. A principal intenção de uma SE é apresentar dados ao usuário por meio de outra lógica de processamento que não apenas a recuperação de dados ou informações de controle. A lógica de processamento deve conter fórmula matemática ou cálculo, criar dados derivados, manter um ou mais ALI e/ou alterar o comportamento do sistema. A Figura 4 mostra um exemplo de SE.

Figura 4 – Exemplo de saída externa

Fonte: elaborada por Rattmann, 2024, com base em Costa, S.d.

Cada funcionalidade identificada é classificada de acordo com sua natureza e complexidade, baseada na contribuição que cada funcionalidade oferece para o usuário e no impacto que tem no sistema como um todo. Por exemplo, uma funcionalidade que permite ao usuário realizar uma operação complexa pode ser classificada como mais "pesada" em termos de pontos de função do que uma funcionalidade mais simples.

As funcionalidades são pontuadas com base em uma escala predefinida, que reflete a sua complexidade e impacto conforme a pontuação determinada de acordo com diretrizes da CPM, como mostra a Figura 5 para funções de transação e para os arquivos lógicos.

Figura 5 – Avaliação da complexidade das funções

Tabela de complexidade funcional dos ALI e AIE

		Tipos de Dados		
		< 20	20 – 50	> 50
Tipo de Registros	1	Baixa	Baixa	Média
	2 – 5	Baixa	Média	Alta
	>5	Média	Alta	Alta

(FONTE: VAZQUEZ, SIMÕES e ALBERT, 2006, p. 77)

Tabela de contribuição dos PF não-ajustados das funções do tipo dado

Tipo de Função	***Baixa***	***Média***	***Alta***
Arquivo lógico interno	7 PF	10 PF	15 PF
Arquivo de interface interna	5 PF	7 PF	10 PF

(FONTE: VAZQUEZ, SIMÕES e ALBERT, 2006, p. 80)

Tabela de contribuição dos PF não-ajustados das funções do tipo transação

Tipo de Função	***Baixa***	***Média***	***Alta***
Entrada Externa	3 PF	4 PF	6 PF
Saída Externa	4 PF	5 PF	7 PF
Consulta Externa	3 PF	4 PF	6 PF

(FONTE: VAZQUEZ, SIMÕES e ALBERT, 2006, p. 112)

Ao final do processo, o total de pontos de função é calculado somando as pontuações atribuídas a todas as funcionalidades identificadas. Esse total representa o tamanho funcional do *software*, proporcionando uma medida quantitativa do valor que o sistema oferece aos seus usuários.

É um processo centrado no usuário, que busca entender as funcionalidades importantes para ele, e como elas contribuem para a eficácia do sistema, fornecendo uma medida objetiva do tamanho funcional do *software*.

2.2 Adaptação da técnica de FPA para metodologia ágil

Agora, vamos avaliar como adaptar a técnica de estimativa como Pontos de Função (PF) para medir um *software* desenvolvido com metodologias ágeis. Vamos considerar os seguintes passos:

- **Compreensão do contexto do projeto:** Antes de começar a adaptar a técnica, é importante entender completamente o contexto do projeto. Isso inclui a compreensão das necessidades do cliente, o escopo do projeto, os requisitos funcionais e não funcionais e a equipe de desenvolvimento envolvida. Lembre-se de que os requisitos são essenciais para uma boa avaliação de pontos de função.
- **Identificação das funcionalidades:** Utilizando a técnica de PF, a equipe de desenvolvimento identifica as funcionalidades do *software*. Isso pode ser feito pela decomposição dos requisitos do projeto em funções individuais que agregam valor para o usuário.
- **Determinação dos pontos de função:** A equipe atribui pontos de função a cada funcionalidade identificada com base em sua complexidade e tamanho. Os pontos de função são determinados por meio de uma fórmula que leva em consideração os seguintes fatores:
  - Número de entradas de dados;
  - Número de saídas de dados;
  - Número de consultas ao sistema;
  - Número de arquivos lógicos internos;
  - Número de arquivos de interface externa.
- **Estimativa dos pontos de função por iteração/*sprint*:** É nesse momento que precisamos adaptar o processo de medição, pensando no processo ágil. Em vez de estimar todos os pontos de função do *software* de uma vez, a equipe de desenvolvimento pode optar por estimar os pontos de função por iteração ou *sprint*. Isso ajuda a tornar a estimativa mais granular e adaptável às mudanças no decorrer do projeto.
- **Iteração e *feedback* contínuos:** Durante o desenvolvimento ágil, é essencial iterar sobre as estimativas de pontos de função à medida que novas informações e requisitos surgem. A equipe deve estar aberta ao *feedback* contínuo do cliente e aos ajustes necessários nas estimativas de pontos de função.
- **Monitoramento e controle:** Ao longo do projeto, a equipe monitora e controla os pontos de função acumulados em cada iteração/*sprint* para garantir que o desenvolvimento esteja alinhado com as expectativas do cliente e com o cronograma do projeto.

- **Revisão e aprendizado:** Após a conclusão do projeto, a equipe realiza uma revisão para avaliar a precisão das estimativas de pontos de função e identificar áreas para melhoria. Isso ajuda na aprendizagem contínua e no aprimoramento das técnicas de estimativa para projetos futuros.

Adaptar uma técnica de estimativa como Pontos de Função para projetos ágeis requer flexibilidade, colaboração e uma abordagem iterativa. A experiência da equipe também é importante.

TEMA 3 – MODELOS DE ESFORÇO: *CONSTRUCTIVE COST MODEL* (COCOMO)

Vamos explorar também o *Constructive Cost Model* – COCOMO ("Modelo de Custo Construtivo"), um modelo de estimativa de esforço amplamente reconhecido na indústria de *software*. Investigaremos suas várias versões e como podem ser usadas para calcular o esforço e os recursos requeridos em projetos de desenvolvimento de *software*.

3.1 Como funciona o modelo COCOMO

Acessaremos uma breve visão geral da origem e evolução do COCOMO e seus conceitos básicos, incluindo a relação entre tamanho do *software*, esforço de desenvolvimento e tempo necessário para conclusão. O modelo considera diferentes fatores, como a complexidade do projeto, a experiência da equipe e a maturidade dos processos. Há variantes do COCOMO, como COCOMO I, COCOMO II e COCOMO III, e suas respectivas características e aplicações. Abordaremos a mais atual.

- **Evolução histórica do COCOMO:** o COCOMO surgiu como uma resposta à necessidade da indústria de *software* de uma abordagem mais sistemática e precisa para estimar o esforço e os recursos necessários em projetos de desenvolvimento de *software*. Desenvolvido por Barry Boehm e sua equipe de pesquisa na década de 1980, o COCOMO representou um avanço significativo na área de engenharia de *software*. Boehm, um renomado pesquisador e professor da área na Universidade do Sul da Califórnia, liderou um esforço para desenvolver um modelo de estimativa que pudesse levar em consideração a complexidade variável

dos projetos de *software* e os fatores que influenciam o esforço de desenvolvimento.

- **Variantes do COCOMO:** o COCOMO evoluiu ao longo dos anos, resultando em diferentes variantes do modelo que são adaptadas para diferentes contextos e necessidades. A primeira versão do COCOMO, conhecida como COCOMO I, foi introduzida por Boehm em 1981, e baseava-se em equações lineares simples para estimar o esforço e o tempo em projetos de desenvolvimento de *software*. Posteriormente, o COCOMO II foi desenvolvido para levar em consideração uma gama mais ampla de fatores e variáveis que influenciam o esforço e o tempo em projetos de *software*. Essa versão do modelo é mais flexível e pode ser personalizada para atender às necessidades específicas de diferentes tipos de projetos. Além disso, o COCOMO III foi introduzido como uma extensão do COCOMO II, incorporando novos *insights* e técnicas de modelagem para melhorar a precisão das estimativas. Essa versão do modelo é ainda mais sofisticada e oferece uma gama mais ampla de opções para estimar o esforço e os recursos em projetos de desenvolvimento de *software*.
- **Princípios fundamentais do COCOMO:** o COCOMO baseia-se em alguns princípios fundamentais para sua aplicação eficaz. Em primeiro lugar, o modelo reconhece que o tamanho do *software* é um fator importante na determinação do esforço necessário para desenvolvê-lo. Isso significa que projetos maiores tendem a exigir mais esforço e recursos do que projetos menores.

Além disso, o COCOMO leva em consideração a complexidade do projeto, incluindo fatores como requisitos de desempenho, restrições de tempo e orçamento e incerteza técnica. Projetos mais complexos geralmente exigem mais esforço e tempo para serem concluídos do que projetos mais simples.

Além disso, o COCOMO reconhece a importância da experiência da equipe e da maturidade dos processos na determinação do esforço necessário para desenvolver o *software*. Equipes mais experientes e processos mais maduros tendem a ser mais eficientes e produtivos, o que pode reduzir o esforço necessário para concluir um projeto (Boehm, 2000).

3.2 O Modelo COCOMO

O COCOMO baseia-se em um conjunto de equações matemáticas que relacionam o tamanho do *software*, a complexidade do projeto e outros fatores relevantes ao esforço de desenvolvimento necessário e com variantes, cada uma adaptada para diferentes tipos de projetos e níveis de detalhe.

O primeiro passo ao utilizar o COCOMO é identificar o tamanho do *software* em termos de linhas de código, pontos de função ou outra métrica relevante. Isso pode ser feito com base nos requisitos do projeto e na análise dos artefatos de *software* disponíveis. Depois, selecionar o modelo COCOMO mais apropriado para o projeto em questão, dependendo das características do projeto e da disponibilidade de dados. Com o modelo COCOMO selecionado, o próximo passo é coletar dados relevantes sobre o projeto, como requisitos, especificações técnicas e características do ambiente de desenvolvimento que são inseridos no modelo COCOMO, que realiza cálculos para estimar o esforço, custo e tempo necessários para o projeto. Uma vez que o modelo COCOMO tenha gerado as estimativas iniciais, os resultados são analisados e refinados conforme necessário, ajustados, revisados e as estimativas geradas pelo COCOMO são comunicadas às partes interessadas, como clientes, gerentes de projeto e equipe de desenvolvimento. É importante apresentar as estimativas de forma clara e compreensível, destacando as principais suposições e limitações do processo de estimativa.

Figura 6 – COCOMO obtido por dados históricos

Vamos analisar, a seguir, como é a aplicação prática do modelo COCOMO. É um modelo empírico, obtido por meio de dados históricos que foram coletados e analisados para descobrir fórmulas que os representem e incorpora quatro submodelos que produzem estimativas cada vez mais detalhadas (Figura 6):

- **Modelo de composição de aplicação:** usado quando o *software* é composto de partes existentes e, também usado na estimativa de protótipos. A estimativa é feita em pontos de aplicação, considerando o número de telas, relatórios, módulos e scripts de BD. Este método aplica uma fórmula que avalia o esforço de cada pessoa envolvida na equipe, o número de pontos de função do *software*, percentual de reuso de trechos de códigos e a produtividade de ponto de aplicação por desenvolvedor. Isto significa que uma equipe menos experiente, tende a ser menos produtiva e isto é utilizado na estimativa do *software*. Observe que, para utilizar este modelo, se faz necessário conhecer o histórico de desenvolvimento da equipe e obter uma contagem de pontos de função.
- **Modelo de projeto preliminar:** usado quando os requisitos foram levantados, mas o projeto ainda não foi iniciado. O objetivo é uma estimativa aproximada de custo baixo e realizada em pontos de função, depois convertida para linhas de código, considerando o número de entradas, saídas, interações com o usuário, interfaces externas e arquivos. Para efetuar um cálculo, utiliza parâmetros como a capacidade pessoal da equipe, complexidade, reuso, dificuldades da plataforma, prazos e outros.
- **Modelo de reuso:** usado para calcular o esforço de integração de componentes reusáveis e usado em conjunto com o modelo pós arquitetura. Envolve a avaliação dos custos de compreender o código, de avaliação e tomada de decisão e da adaptação do código para reuso.
- **Modelo de pós-arquitetura:** usado quando já foi projetada a arquitetura de sistema e as informações adicionais sobre o sistema estão disponíveis. A estimativa é feita usando a fórmula geral com 17 multiplicadores que refletem a capacidade de pessoal e características do produto e projeto.

No mercado, existem ferramentas para a utilização que realizam o cálculo de métricas e estimativas a partir do modelo COCOMO. A escolha da ferramenta

depende das necessidades específicas da organização e da complexidade do projeto de *software* em questão.

Como vimos, o COCOMO não é apenas uma ferramenta de cálculo, mas também uma metodologia que promove uma compreensão mais profunda dos fatores que impactam o desenvolvimento de *software*. Temos que considerar uma ampla gama de variáveis, desde o tamanho e a complexidade do projeto até as características da equipe de desenvolvimento e as restrições do ambiente e, ao aplicarmos o COCOMO, somos capacitados a tomar decisões mais informadas e alocar recursos de forma mais eficaz e a evitar surpresas desagradáveis ao longo do ciclo de vida do projeto, ao mesmo tempo em que nos permite comunicar de forma mais clara e transparente com as partes interessadas.

TEMA 4 – MODELOS BASEADOS NA NESMA

Vamos avançar em nossa análise examinando os modelos de estimativa fundamentados na *Netherlands Software Metrics Users Association* (NESMA, Associação de Métricas de *Software* da Holanda), uma entidade comprometida em impulsionar o uso de métricas de *software* para aprimorar a gestão e a qualidade dos projetos de desenvolvimento. Exploraremos de que maneira esses modelos podem agregar valor e aprofundar as práticas convencionais de estimativa (NESMA, 2024).

A NESMA é uma organização de usuários que se dedica a oferecer alternativas de contagem, utilizando a Análise de Pontos de Função (APF) de modo a permitir a medição de um produto de *software* desde o início do processo, mesmo quando todas as informações sobre as funções de dados, transações e características gerais do sistema ainda não estão totalmente definidas. Essa abordagem possibilita uma avaliação inicial do tamanho funcional do *software*, fornecendo *insights* valiosos para o planejamento e o desenvolvimento do projeto. Compartilhando objetivos e ações similares com o IFPUG, mantém uma colaboração estreita entre ambas as organizações. A associação publicou seu próprio manual de contagem, atualmente na versão 2.1, lançado pela primeira vez em 1990, baseado no manual do IFPUG. Embora os manuais compartilhem de mesma filosofia, conceitos, termos e regras, apresentam algumas diretrizes distintas. As organizações também adotam

diferentes estratégias na aplicação da análise de pontos de função em projetos de melhoria de *software*, detalhadas em seus respectivos documentos.

A NESMA é reconhecida como um dos principais grupos de usuários de métricas de *software*, contribuindo significativamente para o desenvolvimento e aprimoramento das práticas de medição na indústria de *software*.

4.1 Contagem de pontos de função com a NESMA

A contagem de pontos de função com a NESMA pode ser realizada por três métodos distintos: detalhado, estimativo e indicativo (Vazquez, 2003).

O método detalhado segue as mesmas orientações do método detalhado do IFPUG (NESMA, 2024). A contagem detalhada é a contagem usual de pontos de função, e é realizada da seguinte forma:

- Determinam-se todas as funções de todos os tipos (ALI, AIE, EE, SE, CE);
- Determina-se a complexidade de cada função (baixa, média, alta);
- Calcula-se o total de pontos de função não ajustados.

Os métodos indicativo e estimativo são conhecidos por serem contagens antecipadas, pois permitem uma avaliação preliminar do tamanho funcional do *software*. O método indicativo, em particular, pode ser aplicado mesmo antes do início do levantamento completo dos requisitos, fornecendo uma visão inicial do escopo do projeto. Por outro lado, o método estimativo pode ser utilizado com um levantamento parcial dos requisitos, permitindo uma estimativa inicial do esforço e do tamanho do *software* com base nas informações disponíveis até o momento. Essas abordagens antecipadas são valiosas para dar aos *stakeholders* uma ideia inicial do tamanho e da complexidade do projeto, possibilitando melhores tomadas de decisão desde as fases iniciais do desenvolvimento do *software*.

4.1.1 Método indicativo

A contagem indicativa da NESMA considera apenas os arquivos lógicos do *software*, atribuindo um valor fixo para os arquivos lógicos internos e outro valor fixo para os arquivos de interface externa (Vazquez, 2003). Possibilita a estimativa de tamanho a partir de um único componente, os dados ALI e AIE, derivando o resto da contagem a partir de bases estatísticas.

As funções do tipo dado são as que atendem as necessidades de armazenamento de dados do sistema. São classificados em ALI e AIE. Um ALI é um grupo de dados logicamente relacionados, identificável pelo usuário mantido inteiramente dentro da fronteira da aplicação, por meio de Entradas Externas (EE). Já um AIE é um conjunto de dados logicamente relacionado com significado para o usuário e mantido totalmente fora da fronteira da aplicação sendo contada.

Por esse motivo, não é necessário que as funcionalidades do *software* sejam conhecidas, mas apenas os grupos lógicos de dados que serão tratados pelo sistema.

Na Contagem Indicativa, é necessário apenas identificar as funções de dados, ou seja, ALIs e AIE; após isso, o número de PF é indicado de acordo com a fórmula a seguir:

PF = 35 * N° de ALIs + 15 * N° de AIEs

Nessa abordagem, considera-se **35** Pontos de Função para cada **ALI** e **15** Pontos de Função para cada **AIE** identificado.

Os números 35 e 15 representam as médias de pontos de função identificadas para cada um dos tipos de arquivo considerando projetos que, em média, possuam três Entradas Externas (EE), duas Saídas externas (SE) e uma Consulta Externa (CE) para cada ALI e uma Saída Externa (SE) e uma consulta Externa (CE) para cada AIE. Dessa forma, obtém-se uma estimativa ágil do tamanho do sistema, porém não muito precisa.

4.1.2 Método estimativo

A contagem estimativa da NESMA abrange tanto as funções de tipo dado quanto as funções de tipo transação do *software*. É importante notar que, de acordo com essa abordagem, todas as funções de tipo dado são consideradas de baixa complexidade, enquanto todas as funções de tipo transação são tratadas como de complexidade média.

Essa característica torna a contagem estimativa da NESMA uma metodologia antecipada, pois não exige um levantamento completo de requisitos do *software* para ser realizada. Com base nos grupos e subgrupos lógicos de dados e nas funcionalidades do *software* identificadas, é possível obter uma

estimativa do tamanho funcional, independentemente do número de campos envolvidos em cada função.

A contagem estimada é utilizada quando é possível identificar as funções do sistema, porém não se define a complexidade (tipos de dados, tipos de registros e arquivos referenciados). Dessa forma, assume-se uma complexidade padrão para as funções, sendo as funções de dados (ALIs e AIEs) classificadas como de baixa complexidade, e as funções transacionais (EEs, CEs e SEs) são classificadas como de média complexidade.

A contagem estimada é realizada da seguinte forma:

- Determinam-se todas as funções de todos os tipos (ALI, AIE, EE, SE,CE);
- Toda função do tipo dado (ALI, AIE) tem sua complexidade funcional avaliada como baixa;
- Toda função transacional (EE, SE, CE) é avaliada como de complexidade média;
- Calcula-se o total de pontos de função não ajustados

Para realizar o cálculo do tamanho estimativo de um *software* com pontos de função segundo a NESMA, deve-se utilizar a seguinte fórmula:

$$PF = \Sigma ALI*(7PF) + \Sigma AIE*(5PF) + \Sigma EE + \Sigma CE*(4PF) + \Sigma SE*(5PF)$$

De acordo com a Figura 7 e com as regras da Contagem Estimativa da NESMA, um projeto de *software* com 2 ALIs e 1 AIE contribuirá com 19 PF para a contagem. Isso acontece porque o método da NESMA considera todas as funções de dados com complexidade Baixa, logo 2 ALI somam 14 PF e 1 AIE soma 5 PF (Cunha, 2012).

Figura 7 – Pontos de função por tipo dado

Tabela de contribuição dos PF das funções do tipo dado

Tipo de Função	*Baixa*	*Média*	*Alta*
Arquivo lógico interno	7 PF	10 PF	15 PF
Arquivo de interface interna	5 PF	7 PF	10 PF

(FONTE: VAZQUEZ, SIMÕES e ALBERT, 2006, p. 80)

Os métodos de estimativa aplicados antes do levantamento completo dos requisitos do *software* podem fornecer *insights* valiosos sobre o resultado da estimativa realizada após o levantamento de requisitos. Essa descoberta pode

ser útil para orientar a tomada de decisões ao longo do processo de desenvolvimento do *software*.

TEMA 5 – COMPARAÇÃO DE MODELOS DE ESTIMATIVA DE *SOFTWARE*

A análise de pontos de função (APF) é uma técnica essencial para medir o tamanho funcional de sistemas de *software* utilizada para estimar esforço, custo e complexidade em projetos de desenvolvimento e manutenção de *software*, e os métodos mais conhecidos para isso são o IFPUG e o NESMA. Embora ambos os métodos compartilhem sua base conceitual, eles possuem diferenças que os tornam adequados para diferentes contextos e necessidades. A principal diferença reside na abordagem, pois o IFPUG é mais detalhado e formalizado, e o NESMA busca ser mais pragmático e simplificado.

As principais técnicas de estimativa de *software* assumem que o tamanho de um *software* é um vetor importante para a determinação do esforço para sua construção. Saber o seu tamanho é um dos primeiros passos do processo de estimativa de esforço, prazo e custo (Vazquez, 2003).

Dadas as características de um ***software* novo**, ou seja, um *software* que ainda será projetado, desenvolvido e implantado, é possível afirmar que as estimativas realizadas antes e depois da especificação de requisitos sempre terão resultados aproximados?

E para um **projeto de melhoria**, como uma manutenção corretiva com mudança de regras de negócios, novas funcionalidades ou a necessidade de implementar um módulo novo? Como medir e ter certeza de que o custo e o esforço em tempo poderão ser assumidos pela equipe interna, ou se deverá ser contratado com uma empresa externa?

Ou ainda, em uma situação em que uma empresa ou cliente precise de um novo projeto de *software*, mas ainda **não tem todas as informações** e requisitos detalhados, mas ainda assim há a necessidade de estimar uma previsão de custo e tempo? Como garantir respostas adequadas?

As respostas a essas questões são de grande importância para as empresas, pois influenciam diretamente suas decisões estratégicas. Falhas nesse processo podem resultar em prejuízos financeiros, atrasos no lançamento de produtos ou serviços, entre outros impactos negativos. Além disso, as estimativas obtidas podem orientar a decisão de adquirir um novo *software* ou

contratar recursos internos para o desenvolvimento, dependendo da viabilidade identificada.

Faremos uma análise comparativa entre as estimativas realizadas pelos métodos do IFPUG e da NESMA para estimar o tamanho funcional do *software* em diferentes etapas do ciclo de vida de um *software*. O método da NESMA será utilizado na estimativa que considera práticas que **antecedem** a especificação de requisitos do sistema, enquanto o método do IFPUG será utilizado na obtenção de uma estimativa que utiliza a especificação de **requisitos** como subsídio.

Um trabalho realizado por Bundschuh e Dekkers (2008) comparou os métodos IFPUG E NESMA não em si, mas com relação aos resultados que suas estimativas indicam, e se são similares ou não. Os resultados revelaram que, embora ambos os métodos usem abordagens diferentes para estimar o tamanho do *software*, suas estimativas tendem a ser comparáveis em muitos casos. No entanto, também foram identificadas diferenças significativas entre os resultados produzidos pelos métodos. Uma das descobertas mais interessantes foi que, em certos contextos e tipos de projetos, um dos métodos pode produzir estimativas mais precisas do que o outro. Isso sugere que a escolha entre IFPUG e NESMA pode depender da natureza específica do projeto e das preferências da equipe de desenvolvimento. Além disso, o estudo destacou a importância de considerar vários fatores, como complexidade do projeto, experiência da equipe e características do *software* para a escolha o método de estimativa mais apropriado.

Em suma, enquanto IFPUG e NESMA diferem em suas abordagens, seus resultados de estimativa tendem a convergir em muitos casos, embora variações significativas possam ocorrer dependendo das circunstâncias individuais do projeto e, como conclusão dos autores da pesquisa, a estimativa tem relação com as características das funcionalidades do *software*.

É necessário avaliar, também, outros fatores, como o objetivo da medição. Se, por exemplo, o valor de pontos de função e o esforço do desenvolvimento for obtido para avaliar se o *software* deverá ser desenvolvido ou comprado, pode ser que uma estimativa inicial seja suficiente. No entanto, se a principal necessidade for ter certeza do tempo e do custo para optar entre mais de um *software* ou, caso haja uma urgência na escolha com um orçamento delimitado, então é possível que seja necessário um cálculo mais cuidadoso sobre os

requisitos já elicitados. Cabe ao gerente do projeto avaliar em conjunto com o cliente sobre o tipo de contagem mais adequado para aquele caso.

Nesse nosso estudo, o que precisamos conhecer é que há técnicas de estimativa nos três contextos: sobre um **projeto novo** (*software* não existe ainda): mede a funcionalidade fornecida aos usuários do *software* no momento da sua primeira instalação; **projeto de melhoria** (manutenção): mede as funções adicionadas, modificadas ou excluídas do sistema; **aplicação** (existente): mede a funcionalidade fornecida aos usuários por uma aplicação instalada, tratando-se da medição detalhada e completa daquele *software* (geralmente efetuada ao final da implantação).

5.1 Passos para a análise de pontos de função pelo IFPUG

Vamos avaliar um exemplo de necessidade para exemplificarmos o método de contagem de pontos de função pelo IFPUG.

O exemplo trata de um sistema de cadastro de livros, usuários e fornecedores, contendo funcionalidades para incluir, consultar e emitir relatórios como os principais requisitos identificados.

Para isso, consideramos que o novo *software* precisará de interação com entidade externa (AIE,) que serão os **fornecedores** dos livros para obter informações. Como ALIs, ou seja, os dados que serão mantidos, armazenados, manipulados e atualizados pelo próprio sistema, identificamos pelos requisitos, que são **livros** e os **usuários**, e todos os tipos de dados (TD) e os tipos de registros (TR) de cada entidade, gerando assim uma lista com a descrição do ALI, de seus tipos de dados e quantidades (exemplo de TD: código do livro, título do livro, autor do livro).

Ao seguir a tabela de complexidade, determinamos a **complexidade** de cada ALI e associamos à quantidade de pontos de função.

O próximo passo é realizar a contagem das funções do tipo transação (**EE**, **SE**, **CE**) e, para isso, os requisitos funcionais precisam estar mais detalhados e com a explicitação de todas as funcionalidades que comporão o *software*. Por estar no início do projeto e ainda não existirem protótipos de telas, assumimos algumas premissas sobre as funcionalidades para ser possível analisar as complexidades e respectivos pontos de função, como: um botão de **ação** (salvar, excluir, incluir) e uma **mensagem** de retorno ao usuário (são contados como 2 TDs). Depois, identificamos todas as EEs e, listadas de acordo

com sua complexidade, o total de TD e AR (arquivos relacionados) para obter o valor de ponto de função.

A Tabela 1 apresenta o resumo deste método de contagem.

Tabela 1 – Exemplo de contagem de pontos de função pelo IFPUG

Tipo	Função	TD	ALR	Complexidade
EE (entrada externa)	Cadastro de livros	1	2	Média
EE (entrada externa)	Cadastro de usuários	8	1	Baixa
SE (saída externa)	Relatório de empréstimos	15	3	Alta
CE (consulta externa)	Consulta de livros	7	2	Baixa
CE (consulta externa)	Consulta de usuários	10	2	Média
ALI (Arquivo Lógico Interno)	Livros	100	10	Média
ALI (Arquivo Lógico Interno)	Usuários	50	8	Baixa
AIE (Arquivo de Interface Externa)	Fornecedores	20	5	Baixa

Considerando as complexidades e os pontos de função para cada uma, vemos os valores na Tabela 2.

Tabela 2 – Valor calculado de pontos de função do exemplo – 43 pontos de função

Tipo	Função	Complexidade	FP
EE (entrada externa)	Cadastro de livros	Média	4
EE (entrada externa)	Cadastro de usuários	Baixa	3
SE (saída externa)	Relatório de empréstimos	Alta	7
CE consulta externa)	Consulta de livros	Baixa	3
CE (consulta externa)	Consulta de usuários	Média	4
ALI (Arquivo Lógico Interno)	Livros	Média	10
ALI (Arquivo Lógico Interno)	Usuários	Baixa	7
AIE (Arquivo de Interface Externa)	Fornecedores	Baixa	5

5.2 Passos para a análise de pontos de função pela NESMA

Agora, vamos analisar os passos a seguir para efetuarmos a análise de pontos de função pelo método da NESMA. Observe que o método NESMA de estimativa rápida é uma versão simplificada do método completo, utilizando fatores de ajuste e estimativas baseadas em **médias históricas** de complexidade para cada tipo de componente funcional. Dessa forma,

consideraremos que os ALIs têm baixa complexidade e as funções de transação têm média complexidade.

É importante observar que o método NESMA é utilizado, nesse caso, sem ter os requisitos totalmente explicitados e, por esse motivo, assume essas bases históricas para obter um cálculo estimado, como apresentado na Tabela 3.

Tabela 3 – Exemplo de cálculo de pontos de função pelo método NESMA: 36 pontos de função

Tipo	**Função**	**Complexidade**	**FP**
EE (entrada externa)	Cadastro de livros	Média	4
EE (entrada externa)	Cadastro de usuários	Média	4
SE (saída externa)	Relatório de empréstimos	Média	4
CE consulta externa)	Consulta de livros	Média	4
CE (consulta externa)	Consulta de usuários	Média	4
ALI (Arquivo Lógico Interno)	Livros	Média	4
ALI (Arquivo Lógico Interno)	Usuários	Baixa	7
AIE (Arquivo de Interface Externa)	Fornecedores	Baixa	5

Para calcular os pontos de função, somamos de acordo com os tipos identificados (componentes).

5.3 Comparação dos cálculos

Para ambos os métodos (NESMA e IFPUG), utilizamos os mesmos componentes e critérios de classificação. Como resultado, obtivemos um total de 43 pontos de função pelo método IFPUG e 26 pontos pelo método NESMA.

É importante avaliar que houve uma variação entre os dois métodos porque houve diferentes interpretações e critérios específicos utilizados em cada método. Para o IFPUG, foram considerados os detalhes dos requisitos como os tipos de dados (TD) e arquivos referenciados (ALR) de cada função de transacional, pois havia um detalhamento maior dos requisitos. Simulando o método NESMA, que é conhecido por ser uma variante mais pragmática e simplificada do IFPUG, em certos cenários, os resultados podem divergir ligeiramente, especialmente em casos complexos nos quais critérios adicionais são aplicados. No momento em que optamos por um tipo de método ou outro, devemos considerar que isso depende dos objetivos. Se houver a necessidade de algo muito mais certeiro, sem possibilidades de falhas, então os requisitos

devem ser muito bem definidos e o método do IFPUG é o mais indicado. No entanto, se o objetivo for uma avaliação preliminar de custo ou esforço para o desenvolvimento, em que seja aceitável um valor estimado, o método NESMA pode ser mais útil, por não ter todos os requisitos detalhados.

Tanto o IFPUG quanto o NESMA disponibilizam abordagens bem estruturadas e confiáveis para a medição do tamanho funcional de sistemas de *software*. A decisão entre utilizar um método ou outro deve considerar o contexto específico do projeto, a necessidade de precisão em oposição ao pragmatismo, e o nível de familiaridade da equipe com cada método. Entender as particularidades de cada abordagem possibilita uma aplicação mais eficaz e adequada às demandas do projeto, resultando em estimativas de esforço e custo mais precisas e confiáveis.

FINALIZANDO

Ao encerrarmos esta etapa, é importante relembrarmos os diversos temas que exploramos ao longo do curso para consolidarmos os conhecimentos adquiridos.

Inicialmente, discutimos o processo de estimativa de *software*, destacando como as metodologias ágeis, como o *Planning Poker*, foram adaptadas para lidar com a incerteza e a dinâmica dos projetos de desenvolvimento.

Em seguida, mergulhamos nos modelos baseados em pontos de função, reconhecendo sua utilidade na medição do tamanho funcional dos sistemas de *software* e na estimativa de esforço e custo. É importante o destaque dado aos modelos de pontos de função por ser uma das métricas mais amplamente utilizadas na indústria para a comercialização de *software*. Durante nossa discussão, exploramos como esses modelos oferecem uma abordagem robusta e padronizada para medir o tamanho funcional de um sistema de *software*, independentemente da tecnologia utilizada em sua implementação.

No decorrer desta etapa, examinamos também o *Constructive Cost Model* (COCOMO), um modelo clássico de estimativa de esforço que proporciona uma estrutura sólida para avaliar os recursos necessários em projetos de *software*. Além disso, estudamos os modelos baseados na NESMA, reconhecendo sua importância na melhoria da gestão e qualidade dos projetos de *software*, por

meio da aplicação de métricas específicas, principalmente quando não temos todos os requisitos explicitados.

Por fim, concluímos nossa jornada comparando os diferentes modelos de estimativa de *software* discutidos, e sugerimos você a se aprofundar seus conhecimentos em cada área específica.

Cada modelo apresentado oferece abordagens únicas e valiosas para lidar com os desafios da estimativa de *software*, e é muito importante explorar mais a fundo esses conceitos para aplicá-los de maneira eficaz em seus próprios projetos.

REFERÊNCIAS

BOEHM, B. et al. ***Software* cost estimation with COCOMO II**. Hoboken: Prentice Hall, 2000.

BUNDSCHUH, M. and DEKKERS, C. **The IT Measurement Compendium**. Berlin: Springer-Verlag, 2008.

CUNHA, C. R.; FILIPAKIS, C. D. Proposta de Utilização de FDD e APF para Melhoria do Processo de *Software*. In: ENCONTRO DE COMPUTAÇÃO E INFORMÁTICA DO TOCANTINS, 14. **Anais**..., 2012, Palmas. Disponível em: http://ulbra-to.br/encoinfo/artigos/. Acesso em: 2 set. 2024.

EARLY Function Point Counting. **Nesma**, 2024. Disponível em: https://nesma.org/themes/sizing/function-point-analysis/early-function-point-counting/. Acesso em: 2 set. 2024.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de *software***: conceitos e práticas. Porto Alegre: AMGH, 2016.

SOMMERVILLE, I. **Engenharia de *software***. 9. ed. São Paulo: Pearson Education do Brasil, 2011.

VAZQUEZ, C. E.; ALBERT, R. M.; SIMÕES, G. S. Análise de pontos de função-medição, estimativas e gerenciamento de projetos de *software*. São Paulo: Saraiva, 2003.

WANDERLEY, E. G. **Aplicação de pontos por função em projetos que usam métodos ágeis**: uma análise comparativa entre abordagens existentes. Dissertação (Mestrado em Ciência da Computação) –Universidade Federal de Pernambuco, Recife, 2015.